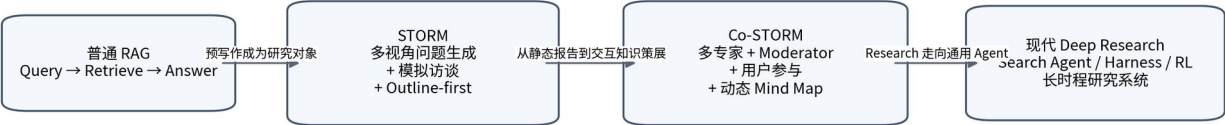


STORM / Co-STORM

完整研究：从多视角预写作到协作式知识策展

Deep Research 系统设计、论文实验、源码架构与可复用 Know-why / Know-how

从 RAG 到 STORM / Co-STORM：研究系统的结构性演化



研究基线：Stanford STORM / Co-STORM 官方论文、stanford-oval/storm 官方源码与当前仓库元数据。研究日期：2026-08-13。

核心判断：STORM 的价值不在“自动写百科”，而在于把 Deep Research 的瓶颈前移到 Pre-writing——先把问题空间探索清楚，再写；Co-STORM 则进一步把静态报告改造成可由人参与、由多 Agent 主动提出未知问题的知识策展过程。

目录与阅读地图

1. 项目定位与时代背景
2. 为什么普通 RAG 不够：Pre-writing 问题
3. STORM 核心架构总览
4. Perspective-Guided Question Asking
5. Simulated Conversation 与搜索循环
6. Outline Generation：知识组织先于写作
7. Section-wise Article Generation 与 Citation
8. FreshWiki 与 STORM 实验体系
9. STORM 的核心 Know-why / Know-how
10. Co-STORM：从静态报告到 Complex Information Seeking
11. Collaborative Discourse Protocol
12. Dynamic Mind Map
13. Moderator：Unknown Unknowns 的主动发现机制
14. WildSeek 与 Co-STORM 实验
15. 源码模块与工程结构
16. STORM vs GPT Researcher / Open Deep Research / DeerFlow / Tongyi
17. 局限、风险与工程改进
18. 自己实现 STORM-style Research Agent 的路线
19. 结论与可累积设计原则
20. 参考资料

阅读建议：若目标是“理解为什么 STORM 重要”，优先读 2→4→6→9；若目标是“实现系统”，优先读 3→5→7→15→18；若目标是“研究未来 Deep Research”，重点读 10→13→16→19。

1. 项目定位与时代背景

STORM 是“研究 + 写作”的系统，不只是一个 Retriever。

STORM (Synthesis of Topic Outlines through Retrieval and Multi-perspective Question Asking) 最初发表于 2024 年 NAACL。论文研究的任务是：在没有预先给定 outline、也没有预先给定 reference documents 的情况下，从一个 topic 出发，自动完成类似 Wikipedia 长文的研究、结构规划和带引用写作。[1]

截至 2026-08-13，官方 GitHub stanford-oval/storm 约有 30,956 Stars、2,899 Forks，MIT License；仓库描述直接将其定位为“研究主题并生成带引用全文报告”的 LLM-powered knowledge curation system。[3]

官方代码随后加入 Co-STORM，并在 knowledge-storm Python 包中提供 STORM 与 Co-STORM 两套 engine。README 说明 v1.1.0 已集成 LiteLLM，retrieval 层支持 YouRM、BingSearch、VectorRM、Serper、Brave、SearXNG、DuckDuckGo、Tavily、Google、Azure AI Search 等多种后端。[3]

从 RAG 到 STORM / Co-STORM：研究系统的结构性演化

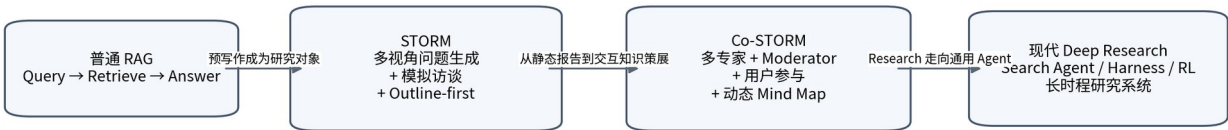


图 1: STORM / Co-STORM 在 Deep Research 演化中的位置 (本报告重绘)

- 传统 RAG 关注“给定 query，怎样找到相关信息”；STORM 关注“一个复杂 topic，到底应该问哪些问题、从哪些视角研究”。
- 传统 long-form generation 往往把 outline 或 references 当输入；STORM 把“找资料 + 做 outline”本身定义为要自动化的核心任务。
- Co-STORM 再进一步指出：复杂信息需求不是一次性 query，而是目标不断变化的 sensemaking / information seeking 过程。

2. 为什么普通 RAG 不够：Pre-writing 是真正瓶颈

STORM 论文把问题拆成两个阶段：第一阶段 “research to generate an outline + collect references”；第二阶段才是基于 outline 和 references 的 full-length writing。[1] 这与人类写作中的 pre-writing / drafting / revising 分阶段过程一致。

Know-why ①：长文质量的上限，往往在写作前就被决定。若 research coverage 不足，后面再强的 writer 也只能把有限证据组织得更漂亮。

Know-why ②：复杂主题不存在唯一理想 query。简单 topic search 通常只返回高频、表层信息，因此“搜索什么”本身需要一个策略。

Know-why ③：LLM 直接生成问题容易偏向 What / When / Where 等表面问题。STORM 的两个假设是：不同 perspective 会产生不同问题；深入问题需要迭代研究，即后一个问题依赖前一个答案。[1]

范式	已给 Outline?	已给 References?	系统真正负责的工作
普通摘要/RAG	常常是	常常是	在有限上下文中生成文本
Outline-driven RAG	是/先生成	搜索补充	按章节写
STORM	否	否	研究问题空间 + 收集证据 + 组织 Outline + 写作

工程意义：对于 Deep Research，必须把“问题分解 / 问题发现 / 证据覆盖 / 知识组织”当成一级组件，而不能只优化最终生成 Prompt。

3. STORM 核心架构总览

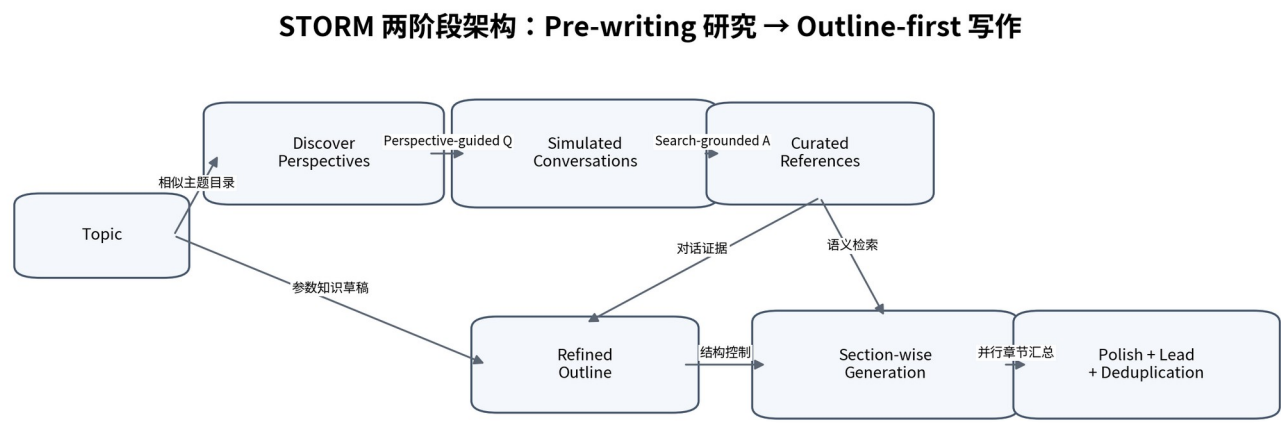


图 2: STORM 两阶段主链路

官方实现可抽象成四个模块：Knowledge Curation、Outline Generation、Article Generation、Article Polishing；接口位于 `knowledge_storm/interface.py`，具体实现位于 `knowledge_storm/storm_wiki/modules/`。 [3]

模块	主要职责	核心输入	核心输出
Knowledge Curation	发现 perspective；模拟问答；检索可信来源	Topic	Conversations + Ref Pool
Outline Generation	先生成 parametric draft outline，再用研究材料 refinement	Topic + Conversations	Hierarchical Outline
Article Generation	按 section 从 Ref Pool 检索相关 evidence 并写作	Outline + Ref Pool	Section drafts
Article Polish	去重、补 lead summary、改善整体呈现	Concatenated article	Final article

注意：STORM 的“多 Agent”不是后来 Supervisor/Subagent 风格的通用 multi-agent runtime。它更接近多个 perspective-conditioned writer 模拟并行研究，系统结构高度针对“知识策展 + 长文写作”任务。

4. Perspective-Guided Question Asking

STORM 首先让 LLM 生成 related topics，再获取这些相似 Wikipedia 页面中的目录结构；这些目录被拼接起来作为 context，用来提取 N 个互补的 perspectives。随后额外加入一个 “basic fact writer” 视角，保证基础事实覆盖。[1]

这个设计的关键不是 Persona 的拟人感，而是把 “搜索空间分区”。Perspective 相当于一种 research prior：它改变下一步会被问到的问题分布。

Topic	Perspective 示例	潜在问题
2022 冬奥会开幕式	Event Planner	交通安排、预算、流程编排、场馆运营
同一主题	政治/外交视角	国际关系、代表团、政治争议
同一主题	技术制作视角	舞台系统、视觉技术、直播与工程实现
同一主题	Basic Fact Writer	时间、地点、参与者、基本事件

Know-how：不要让一个 Planner 一次性生成几十个 “平铺子问题”；先生成少量互补 perspective，再让每个 perspective 在后续对话中递归生长问题，能同时获得 breadth 与 depth。

Know-how：Perspective 的来源最好来自外部结构信号（如相似文档目录、领域 taxonomy、用户 corpus），而不是完全依赖模型凭空 “扮演角色”。这样可以降低角色随机性并提升覆盖稳定性。

5. Simulated Conversation：把检索变成迭代研究循环

STORM 的核心：Perspective-guided + Conversation-driven Question Asking

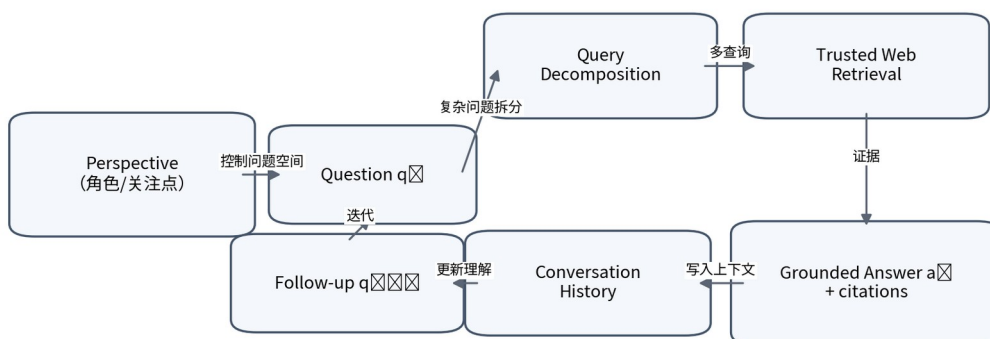


图 3: Question → Search → Grounded Answer → Follow-up 的研究循环

在每个 perspective 下，STORM 模拟 Wikipedia writer 与 topic expert 的多轮对话。第 i 轮问题 q_i 依赖 topic、perspective 以及历史 $q_1, a_1 \dots q_{i-1}, a_{i-1}$ ，因此模型可以基于已知信息提出 follow-up，而不是每次从零生成独立 query。[1]

复杂问题不会直接送给搜索引擎：STORM 先把问题拆成一组 search queries，再进行搜索和可信来源过滤，最后由模拟 expert 基于检索证据合成回答，并把来源写入全局 reference pool。

- Question Asking 与 Retrieval 解耦：研究意图是自然语言问题，执行层可以拆成多个搜索查询。
- Answer 必须 grounded：对话历史之所以能帮助后续提问，是因为历史信息尽量来自外部证据，而不是模型自由发挥。
- Conversation 是一种“可学习的搜索状态”：后续 query 的信息增益取决于先前结果。
- 论文实验中 $N=5$ perspectives, $M=5$ rounds；这是研究预算参数，不应被理解为普适最优值。[1]

与 dzhng/deep-research 的 $\text{breadth} \times \text{depth}$ 递归相比，STORM 的“depth”不是显式树深度，而是 conversation history 驱动的 follow-up depth；其结构更贴近人类访谈式研究。

6. Outline Generation：知识组织先于写作

研究完成后，STORM 不直接让模型“根据所有资料写文章”。它先用 topic 单独生成一个 draft outline O_D，利用模型的 parametric knowledge 提供一个通用结构；然后把 O_D 与所有 perspective conversations 一起输入模型进行 refinement，得到最终 outline O。[1]

这个“Parametric prior + Retrieved evidence”的组合非常重要：完全依赖检索材料容易碎片化，完全依赖参数知识又容易缺少 topic-specific 细节。Draft outline 提供结构先验，research conversations 负责将长尾知识注入结构。

阶段	作用	失败模式
Draft Outline	提供高层组织骨架	可能通用、缺乏细节
Research Conversations	发现长尾、具体、跨视角信息	碎片化、重复、缺少整体结构
Refinement	将新证据映射进结构并扩展/修正章节	依赖模型的归纳与层级组织能力

Know-why：Deep Research 的 Context Engineering 不只是在“压缩 token”，还包括把证据转换为高层结构。Outline 是一种低带宽、强语义的压缩表示，能决定后续检索和写作的路由。

7. Section-wise Article Generation 与 Citation

Outline-first 的写作执行面：按章节局部化 Context

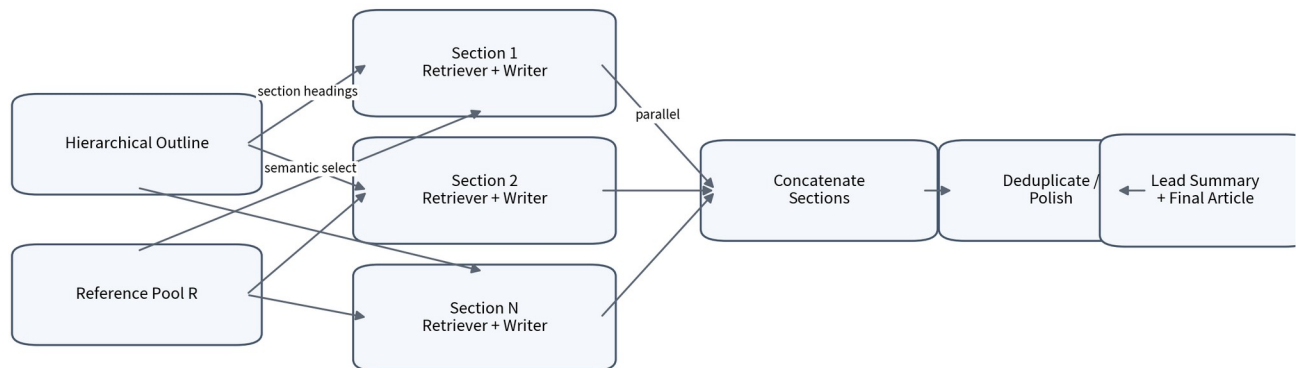


图 4: 按章节局部化 Context 的文章生成

STORM 论文指出整个 Reference Pool 往往无法一次放进 context window，因此对每个 section 使用 section title + subsection headings 作为 query，在 reference pool 中通过 Sentence-BERT semantic similarity 选择相关文档，再单独生成该章节。[1]

每个 section 可以并行生成，最后拼接；由于并行章节会重复信息，再由一个 polishing step 删除重复，并生成全篇 lead summary。

这是一种早期、非常清晰的 Context Partitioning：不是把全部 research context 压缩成一段 summary，而是让 outline 节点承担 routing key，将证据按章节局部化。

- 优势：每个 writer 只看到与章节相关的 evidence，降低 irrelevant context 和注意力稀释。
- 优势：章节级并行，吞吐高。
- 代价：跨章节依赖、全局叙事、引用去重需要后处理。
- 风险：如果 Outline 本身遗漏重要主题，后续 section retrieval 无法补回“从未创建的章节”。

8. FreshWiki 与 STORM 实验体系

为了降低模型训练数据泄漏，STORM 构造 FreshWiki：重点选择 2022-02 至 2023-09 期间新建或大幅编辑、且质量达到 B-class 及以上的 Wikipedia 页面，并过滤列表型文章和没有 subsection 的页面。[1]

论文将 pre-writing 单独评测，提出 Heading Soft Recall 与 Heading Entity Recall：前者用 Sentence-BERT 判断生成 heading 与人工 heading 的软匹配，后者统计人工 heading 中实体被生成 outline 覆盖的比例。[1]

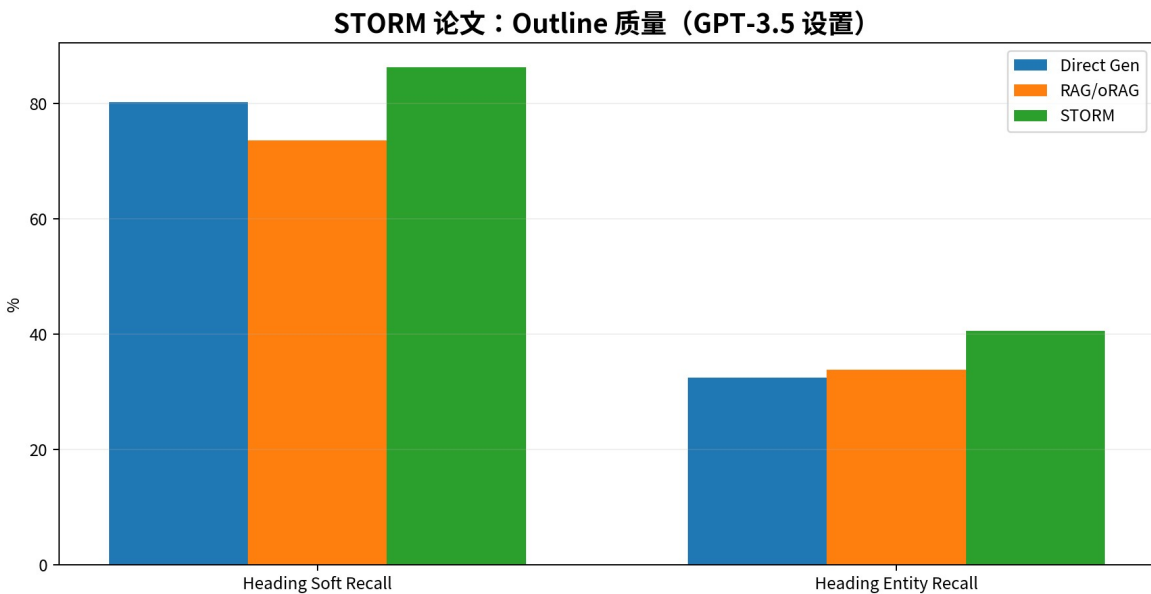


图 5: 论文 Table 3 中 GPT-3.5 设置的 Outline 指标

系统	ROUGE-1	Entity Recall	Interest	Organization	Relevance	Coverage
Direct Gen	25.62	5.08	2.87	4.60	3.10	4.16
RAG	28.52	7.57	3.14	4.22	3.05	4.08
oRAG	44.26	12.57	3.90	4.79	4.09	4.70
STORM	45.82	14.10	3.99	4.82	4.45	4.88

关键实验结论：oRAG 已经证明“Outline-driven section retrieval”明显强于简单 RAG；STORM 仍进一步提升 entity recall、relevance 与 coverage。论文报告 STORM citation recall 84.83%、citation precision 85.18%，但作者也指出不支持句子的主要原因包括不恰当推断与不准确改写。[1]

实验方法上的可学习点：不要只用 final answer score 评估 Research Agent。STORM 显式测量 Outline 覆盖、文章组织、coverage、citation 等中间能力，因此更容易定位失败发生在 Research、Organization 还是 Writing。

9. STORM 的核心 Know-why / Know-how

设计原则	Know-why	Know-how
先研究再写	写作质量受证据覆盖上限约束	把 Pre-writing 做成显式 pipeline
Perspective 作为搜索 prior	单一 Planner 容易集中在高频主题	用多视角分区问题空间
对话驱动追问	深入问题依赖已有答案	保留 grounded research history
Outline 作为中间表示	扁平 evidence 难以直接支持长文	先组织层级结构再生成
Section-local Context	全量 refs 超出窗口且噪声大	按 outline node 检索相关 evidence
Evidence provenance	长文需要可核验	研究阶段就保留 URL/证据映射

可以把 STORM 抽象为一个 Information Gain 驱动的研究器：perspective 决定“从哪个子空间探索”，conversation history 决定“下一步什么问题还未知”，retrieval 提供 evidence，outline 将高维 evidence 压缩成后续写作的结构控制信号。

最值得沉淀的思想不是“模拟 Wikipedia Writer”，而是：Research Agent 需要同时维护三个状态——问题空间状态（What to ask）、证据状态（What is known）、结构状态（How knowledge is organized）。

10. Co-STORM：从静态报告到 Complex Information Seeking

Co-STORM 的出发点是 “unknown unknowns”：用户可能连该问什么都不知道。传统 Search / RAG / Chatbot 主要解决 known unknowns，即用户已经能明确表达自己的问题；复杂研究则要求系统主动暴露用户没有想到的信息方向。[2]

Co-STORM 将任务定义为 complex information seeking：多来源、持续交互、最终形成 curated report。论文认为静态 STORM 报告虽然覆盖广，但缺少用户在研究过程中的参与，不能适配逐步变化的目标。[2]

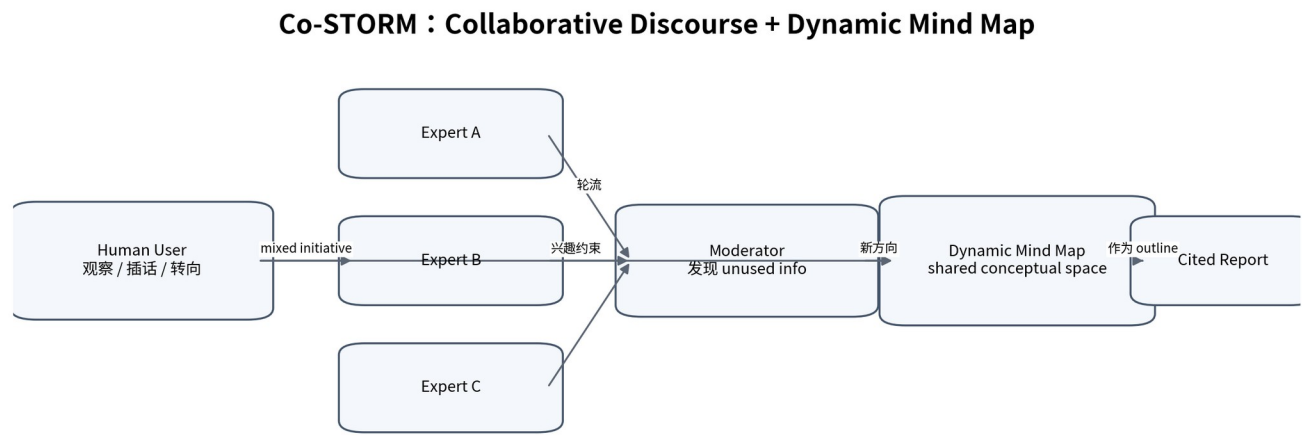


图 6: Co-STORM collaborative discourse protocol

核心范式变化：用户不再承担“连续提出正确问题”的全部认知负担，而可以观察 Experts 与 Moderator 的讨论，在有兴趣或需要纠偏时插入问题或观点。

11. Collaborative Discourse Protocol

Co-STORM 的 discourse turn 来自三类角色：Human User、多个 perspective-guided Experts、Moderator。系统采用 mixed-initiative：用户可随时抢占 turn；用户不参与时，由 Agent 自动推进。[2]

每个专家 utterance 先判断 intent：Original Question、Information Request、Potential Answer、Further Details。前两者属于问问题，后两者属于提供信息。若提供信息，则进行搜索并生成带引用回答；若要提问，则根据 discourse history 直接生成问题。[2]

Turn Management 的工程关键：专家按顺序轮流；如果连续 L 个专家 turn 都只是 Potential Answer / Further Details，Moderator 会介入，避免讨论陷入同一细节分支。论文实验设置 L=2、专家数 N=3。[2]

角色	主要目标	典型失败模式	控制策略
Expert	从特定专业视角深入	越聊越窄、重复补充	多 perspective + intent 分类
Moderator	主动开启新方向	问题太随机/离题	基于 unused information rerank
Human	表达偏好与动态目标	认知负担过高	允许观察为主、偶尔 steer

12. Dynamic Mind Map：把对话转换为共享知识空间

Co-STORM Mind Map 的增量知识组织算法

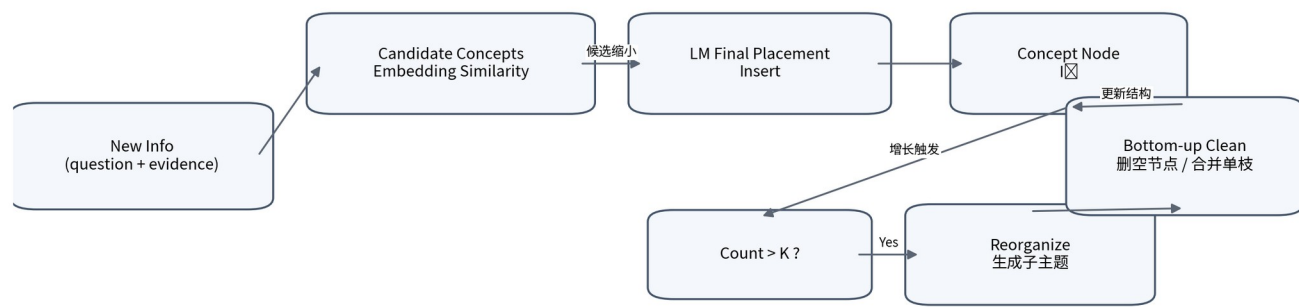


图 7: Mind Map 的 insert/reorganize 增量算法

Co-STORM 用树结构 Mind Map $M=(C,E)$ 动态组织 discourse 中的知识。每个 concept 关联一组 retrieved information I_c ；每条信息还保留“是什么问题触发了这次检索”的关系，因此知识不是纯 chunk，而是 question-conditioned evidence。[2]

Insert：先用 question 与已有 concept 的 embedding similarity 找候选节点，再让 LM 选择最终 placement。

Reorganize：当某 concept 下的信息超过阈值 K 时，LLM 生成新的 subtopics，再重新分配该节点下的 information；之后进行 bottom-up clean，删除空节点、折叠只有一个子节点的结构。论文 $K=10$ 。[2]

Know-why：长对话最大的 Context 问题不只是 token 太多，而是“用户和 Agent 都失去共同结构”。Mind Map 是面向交互的 externalized memory，它同时承担导航、压缩、共享状态和最终报告 outline。

Know-how：将 Research Memory 设计成“层级概念节点 + evidence + triggering question + provenance”，比单纯向量库更适合长时程 research。

13. Moderator：主动发现 Unknown Unknowns 的算法

如果只有专家，讨论容易持续在已有问题周围扩展。Co-STORM 的 Moderator 专门寻找“相关但没有被当前问题直接覆盖”的证据，从而制造新方向。

论文对未使用信息 i 进行 rerank: $score = \cos(i,t)^\alpha \times (1 - \cos(i,q))^{(1-\alpha)}$ 。第一项要求信息与总体 topic 相关；第二项奖励信息与当前 question 不相似，即“不是当前问题的直接答案”。实验 $\alpha=0.5$ 。[2]

这其实是一个显式的 Serendipity / Novelty objective：不是最大化 query relevance，而是最大化 Topic relevance 与 Current-question novelty 的组合。

目标函数	优化对象	适合场景
纯检索 relevance	回答当前 query	事实问答、已知 unknown
coverage / diversity	覆盖更多子主题	报告型 Deep Research
topic relevance $\times$ query novelty	暴露用户尚未想到的方向	探索、学习、战略研究

对现代 Research Agent 的启发：Search policy 不应只有“与 query 最相关”一个目标；需要显式引入 novelty、coverage、uncertainty、source diversity 等探索信号。

14. WildSeek 与 Co-STORM 实验体系

Co-STORM 构造 WildSeek：从真实用户在 STORM 网站上的 topic + goal 记录中筛选高质量数据，最后得到 100 个 data points，覆盖 24 个 domains。[2]

自动评测让模拟用户与系统交互，并对所有方法统一限制到 30 search queries；最终报告均采用 STORM 的两阶段 outline → section-wise generation，保证比较主要集中在“信息寻求过程”。[2]

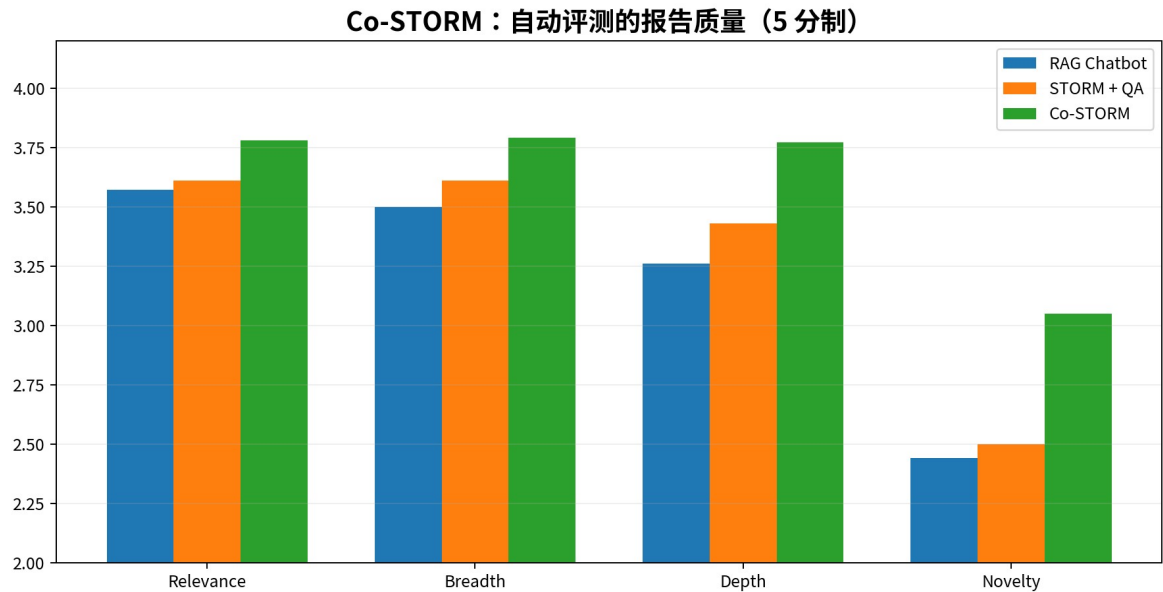


图 8: Co-STORM 自动评测的报告质量

方法	Depth	Novelty	QA Consistency	QA Engagement	Unique URLs
RAG Chatbot	3.26	2.44	4.37	4.13	2.94
STORM + QA	3.43	2.50	4.34	4.11	2.89
Co-STORM	3.77	3.05	4.40	4.33	6.04
w/o Multi-Expert	3.77	2.93	4.40	4.32	5.91
w/o Moderator	3.41	2.89	4.39	4.28	5.67

Ablation 很有价值：移除 Moderator 的负面影响大于把多专家减少为单专家，说明“主动注入新问题方向”的角色比堆更多 agent 数量更重要。[2]

Human Study：20 名参与者；论文报告 70% 总体偏好 Co-STORM 胜过 Search Engine、78% 偏好其胜过 RAG Chatbot。更细的评价中，Co-STORM 的 serendipity 对 Search Engine 为 3.90 vs 2.70，对 RAG Chatbot 为 3.78 vs 2.78；参与者还认为其心智负担更低。[2]

15. 源码模块与工程结构

官方源码并不是一个大而全的 Agent framework，而是围绕 knowledge curation 明确分层。STORM 主线由 engine.py 组织，storm_wiki/modules 下含 persona_generator.py、knowledge_curation.py、retriever.py、outline_generation.py、article_generation.py、article_polish.py、storm_dataclass.py 等模块。[3]

Co-STORM 位于 collaborative_storm/: engine.py 外，modules 中可见 co_storm_agents.py、expert_generation.py、grounded_question_answering.py、grounded_question_generation.py、information_insertion_module.py、knowledge_base_summary.py、warmstart_hierarchical_chat.py 等。[3]

STORM / Co-STORM 在 Deep Research 技术谱系中的位置

系统	搜索动作	问题生成	知识组织	用户交互	最终产物
普通 RAG	一次或少量	用户提供	扁平 Context	问答	短答案
GPT Researcher 类	多查询/递归	Planner/Agent	Research Context	弱交互	报告
STORM	多视角并行	Persona + 对话追问	Outline + Ref Pool	无	Wikipedia-like 报告
Co-STORM	持续检索	专家 + Moderator	动态 Mind Map	强	交互知识策展 + 报告

图 9: 不同 Research 系统的结构关注点

源码层	STORM	Co-STORM
Orchestrator	STORMWikiRunner	CoStormRunner
Agent/Persona	persona_generator	co_storm_agents / expert_generation
Research	knowledge_curation + retriever	grounded QA/Q generation
Knowledge Structure	Outline	Knowledge Base / Dynamic Mind Map
Writing	article_generation + polish	article_generation based on mind map
Interaction	batch run()	warm_start() + step(user_utterance=...)

工程风格：官方 README 强调多 LM 配置，不同模块可用不同模型。例如便宜/快模型用于 conversation simulator、question asker，更强模型用于 outline/article generation。这与现代 Deep Research 中的 Model Routing 思路高度一致。[3]

16. 与 GPT Researcher / Open Deep Research / DeerFlow / Tongyi 的差异

系统	核心控制变量	Research State	知识组织	主要扩展方向
STORM	Perspective + conversation	多视角对话 + Ref Pool	Hierarchical Outline	高质量预写作/报告
Co-STORM	Turn policy + Moderator	Discourse + Evidence	Dynamic Mind Map	Human-in-the-loop sensemaking
GPT Researcher	Sub-query / breadth-depth	Learnings + Context	Report structure	成熟 retrieval/scrape/tooling
LangChain ODR	Supervisor + researcher loop	LangGraph State	Compressed research notes	通用工具、MCP、并行 subagents
DeerFlow 2.0	Lead Agent + harness	Files/Memory/Thread State	Skills/Artifacts	长任务通用 SuperAgent
Tongyi DeepResearch	Learned agent policy	模型内部 trajectory	隐式 reasoning + external evidence	Agentic CPT/SFT/RL

STORM 最大的独特性仍是 “Question Asking + Knowledge Organization”。它不像 LangChain ODR 那样强调通用 Supervisor，也不像 DeerFlow 强调 Sandbox/Harness，更不像 Tongyi 将搜索策略训练进模型。

因此，STORM 最值得被现代 Agent 复用的部分，不一定是整套 engine，而是三个组件：Perspective Generator、Iterative Question-Asking Policy、Outline/Mind-Map Knowledge Organizer。

17. 局限、风险与工程改进

STORM 论文的 Wikipedia 编辑者反馈指出两个关键风险：互联网偏见会传递到生成文章；模型可能把彼此无关的事实错误地“建立联系”。[1] 这两点在长文 Deep Research 中尤其危险，因为文字连贯性可能掩盖证据链的不连续。

Co-STORM 论文进一步列出：需要更好适配用户已有知识；用户希望对专家视角和 utterance length 有更多控制；多语言能力依赖多语言 retrieval；相较 RAG Chatbot，Co-STORM 因 intent decision 和 mind-map update 有额外 latency。[2]

问题	根因	改进建议
Source bias transfer	Web evidence 本身偏置	source diversity / authority scoring / contradiction search
False association	Writer 为追求连贯而过度推断	claim-evidence graph + entailment check
Outline blind spot	没进入 outline 的主题难被后续发现	post-outline gap search / coverage critic
Section inconsistency	并行写作缺少全局状态	global facts ledger / cross-section reviewer
Mind Map drift	增量分类误差累积	周期性 rebuild + deterministic clustering
Latency	多轮 LLM + retrieval + organization	cache / async / smaller routing model

18. 从零实现一个 STORM-style Research Agent

MVP 不需要复刻完整 Wikipedia pipeline。更好的实现顺序是先实现“研究状态机”，再逐步增加写作层。

阶段	最小实现	成功标准
Step 1 Perspective	从 topic 生成 3-5 个非重叠 research perspectives	问题覆盖显著分散
Step 2 Iterative QA	每个 perspective 运行 3-5 轮 search-grounded conversation	后续问题能引用前序发现
Step 3 Evidence Store	保存 URL / title / excerpt / triggering question	可追溯、去重
Step 4 Outline	parametric draft + evidence refinement	章节覆盖关键 evidence
Step 5 Section Router	outline node → top-k evidence	每节上下文小而相关
Step 6 Citation Writer	按 claim 引用来源	citation precision/recall 可评测
Step 7 Gap Critic	检查 coverage / contradiction / missing perspectives	研究可自我补全

一个更现代的版本可以把 STORM 与 LangChain ODR 结合：Supervisor 负责研究预算与并发，Researcher 内部使用 STORM-style perspective/follow-up；Evidence 输出经 compression 后进入全局 Outline/Mind Map；Writer 最后统一生成报告。

如果继续进一步演化，可把 Tongyi 的 Agentic RL 思路用于学习 Search/Question Policy：reward 不只看最终答案，还应加入 evidence coverage、citation correctness、novelty、source diversity、search cost。

19. 结论：STORM 最得累的原

- 研究的第一性问题是“下一步应该问什么”，而不是“下一步调用哪个搜索 API”。
- Breadth 来自多视角；Depth 来自基于已知证据的 follow-up；二者需要不同机制。
- Outline / Mind Map 是 Deep Research 的关键中间表示：它把 Evidence 变成可路由、可导航、可写作的结构。
- Context Engineering 不等于截断历史：更高级的方式是把知识转换为层级结构和 section-local context。
- Multi-Agent 的价值不是 Agent 数量，而是角色是否引入不同目标函数。Co-STORM 的 ablation 表明 Moderator 比“更多专家”更关键。
- Unknown Unknowns 需要探索目标：topic relevance 之外，还应显式优化 novelty / coverage / diversity。
- 研究与写作应分离评测：要分别测问题质量、证据覆盖、结构质量、citation、final report，而不是只看最终文本。
- 未来 Deep Research 的融合方向：STORM 的 Question/Organization + ODR 的 Supervisor/Tool Loop + DeerFlow 的 Harness + Tongyi 的 Learned Policy。

一句话总结：STORM 把“写长文”改写成“先构建足够好的知识结构再写”；Co-STORM 又把“构建知识结构”改写成“人和 Agent 共同探索未知空间”。这两次改写，分别对应 Deep Research 从 RAG 到 Research Workflow、再到 Interactive Knowledge Curation 的关键跃迁。

20. 参考资料与证据边界

本报告将“官方事实 / 论文实验”与“工程推断 / 本报告评价”分开处理。带 [1]-[4] 的内容来自以下原始来源；Know-why / Know-how 中的架构归纳属于基于源码和论文的工程解释，不代表作者原文结论。

[1] Shao et al. (2024), Assisting in Writing Wikipedia-like Articles From Scratch with Large Language Models (STORM), NAACL 2024. <https://arxiv.org/abs/2402.14207>

[2] Jiang et al. (2024), Into the Unknown Unknowns: Engaged Human Learning through Participation in Language Model Agent Conversations (Co-STORM), EMNLP 2024. <https://arxiv.org/abs/2408.15232> ; HTML: <https://arxiv.org/html/2408.15232>

[3] Stanford OVAL, STORM / Co-STORM official repository. <https://github.com/stanford-oval/storm>

[4] Stanford STORM Project. <https://storm-project.stanford.edu/> ; Research preview: <http://storm.genie.stanford.edu>

仓库状态快照（2026-08-13）：约 30,956 Stars、2,899 Forks；default branch main；最新仓库 push 时间在 GitHub API 快照中为 2025-09-30。Star/Fork 属动态指标，未来会变化。[3]

实验数字均以论文报告为准。需要注意：STORM 部分自动 rubric 使用 evaluator LLM；论文作者也明确承认 evaluator 可能对机器文本存在高估，因此不能把自动评分等同于完全独立的人类质量判断。[1]